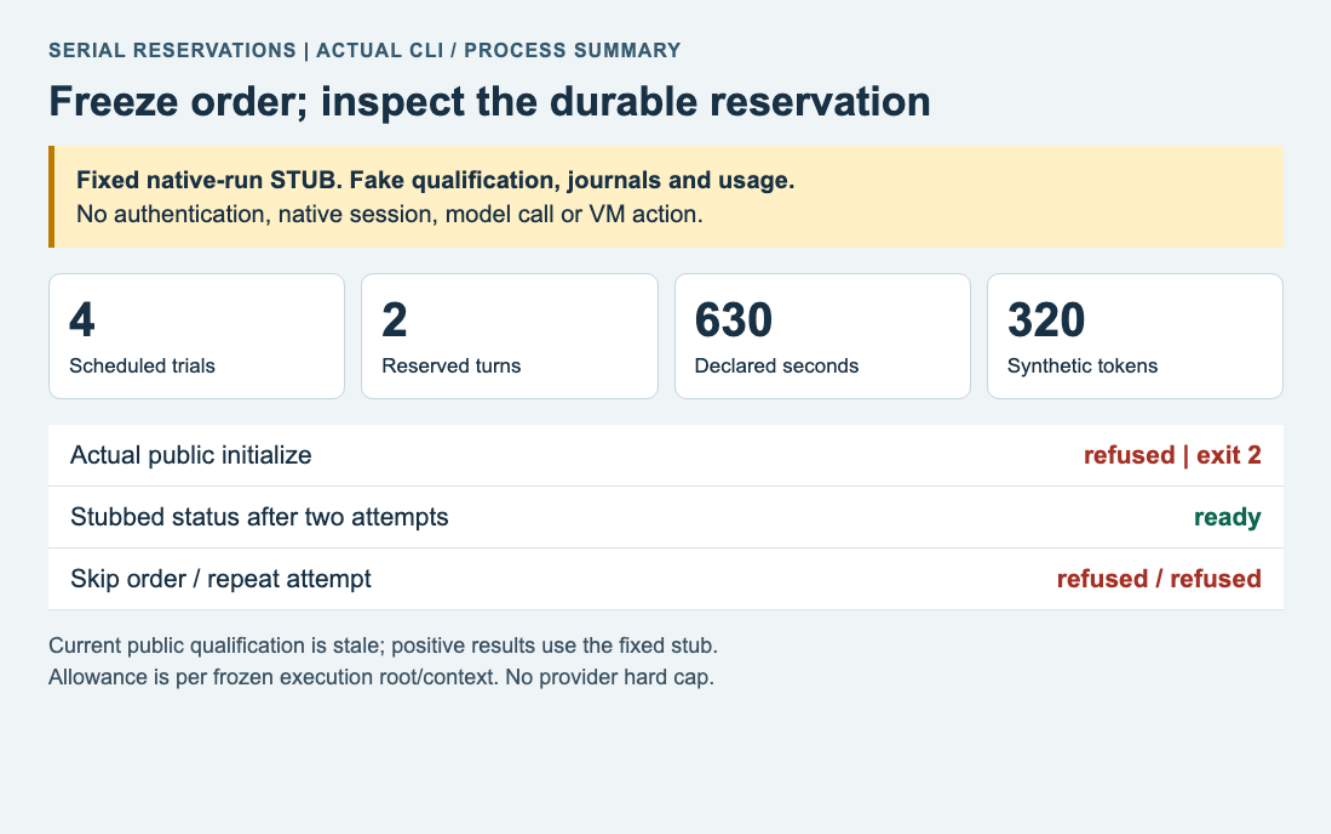
Serial native trial reservations

Evaluation-accounting workflow on Python 3.11+. Browser-rendered actual CLI/process summaries use a fixed native-run STUB and fake qualifications, journals and usage. Not terminal images or native/model evidence.

1. Initialize and follow the frozen order

Read the serial contract before using trial_reservation.py initialize, status or run-next. Supply the frozen stdin context and private execution root. Launch plans in that root only through run-next; direct native run bypasses this gate. Current public qualification refuses initialization; the positive example uses a fixed test-only native-run stub.



Exact bytes from renewed smoke match checkpoint a8b0f9785457 after execution. The commit was created later; this is not execution at that checkpoint. Thirteen runner and three grader hashes match.

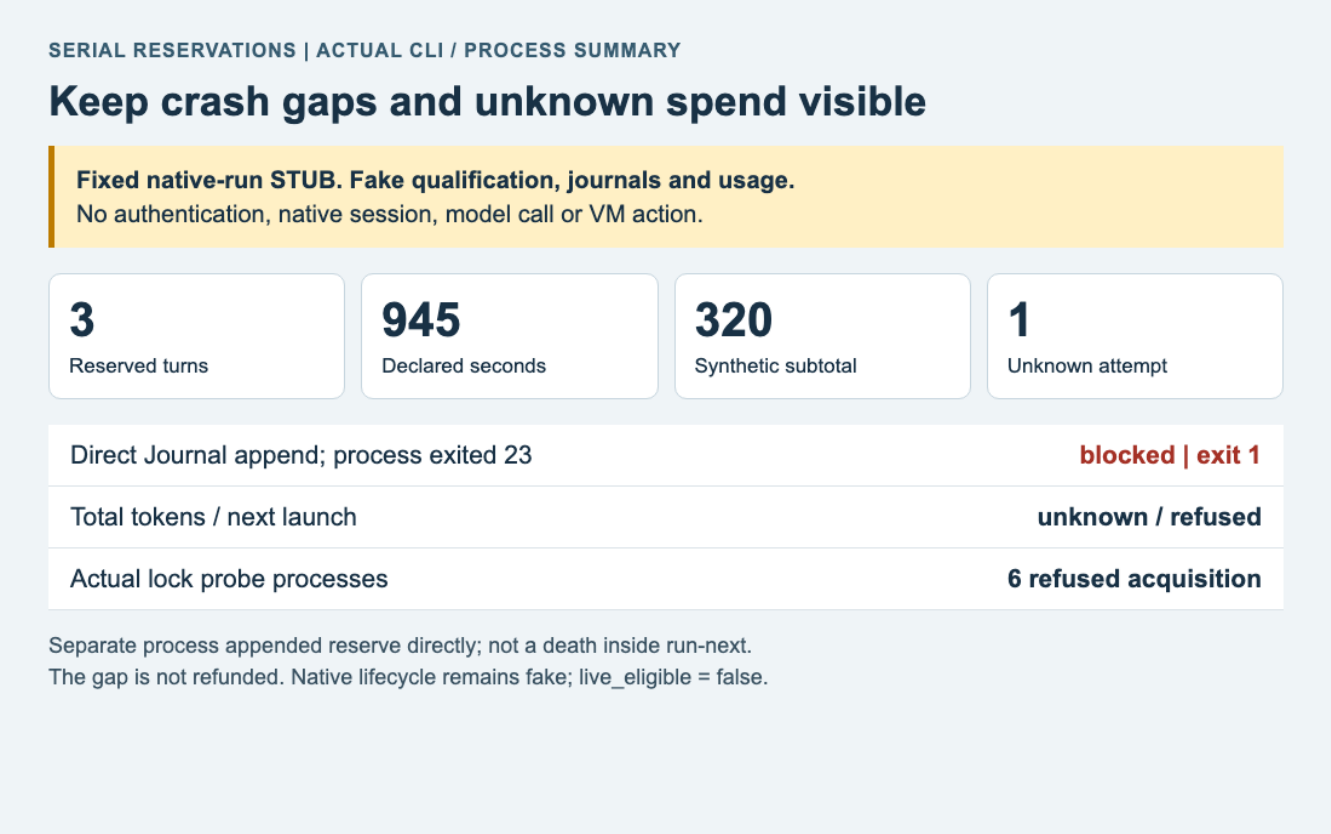
Expected result: After two stubbed attempts: two reserved turns, 630 declared seconds and 320 synthetic tokens. Wrong order and retry refuse. These are initial-turn and permanent phase-time reservations, not provider request counts.

Serial native trial reservations

Evaluation-accounting workflow on Python 3.11+. Browser-rendered actual CLI/process summaries use a fixed native-run STUB and fake qualifications, journals and usage. Not terminal images or native/model evidence.

2. Audit interruption without a refund

Inspect status after interruption and retain the original reservation. Check execution_accounting when completion references omit an attempt. Unknown or incomplete facts prevent another launch; retain every original root/context for combined reporting. A new root/context creates an undetected new allowance.



A separate process directly appended Journal reserve, then exited 23. It did not crash inside run-next. Native lifecycle is synthetic; separate lock probes ran before task and after stubbed cleanup.

Expected result: Three reservations and 945 declared seconds remain. Synthetic 320 tokens are known; total is unknown and the next launch refuses. Post-use soft token checks may overshoot; hard_provider_cap and live_eligible stay false.